

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Mandatory Plots and Outputs

The following plots and outputs were generated during the experiment. Each figure is placed under its corresponding heading.

9.1 Sample CIFAR-10 Images

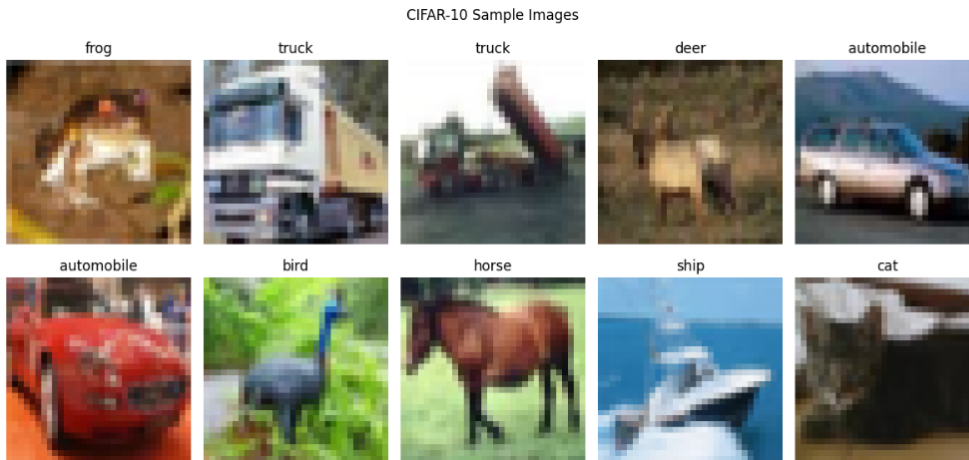


Figure 1: Sample CIFAR-10 Images

Inference: The figure shows ten representative images from the CIFAR-10 dataset. The images belong to different object classes and demonstrate the variety of visual patterns present in the dataset.

9.2 Class Distribution

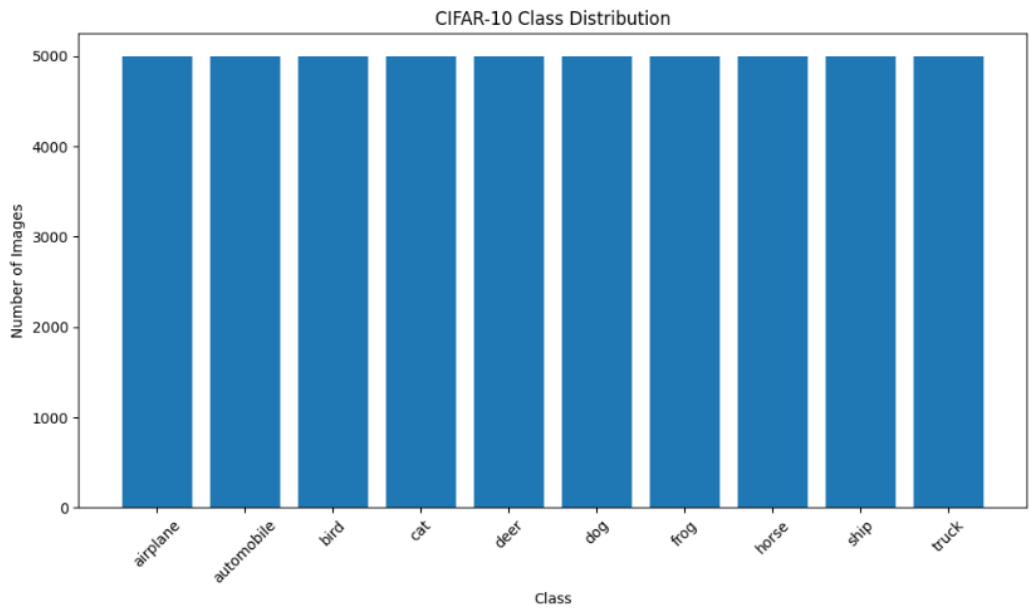


Figure 2: CIFAR-10 Class Distribution

Inference: The class distribution shows the number of images available for each CIFAR-10 class. The dataset is approximately balanced, with a similar number of samples for every class.

9.3 Comparison of Kernel Sizes

```
Using stride = 1 and padding = valid:  
Kernel 3x3 --> Feature map size = 30 x 30 x 16  
Kernel 5x5 --> Feature map size = 28 x 28 x 16  
Kernel 7x7 --> Feature map size = 26 x 26 x 16  
  
Theoretical output dimensions:  
3x3: (32 - 3 + 2(0))/1 + 1 = 30  
5x5: (32 - 5 + 2(0))/1 + 1 = 28  
7x7: (32 - 7 + 2(0))/1 + 1 = 26
```

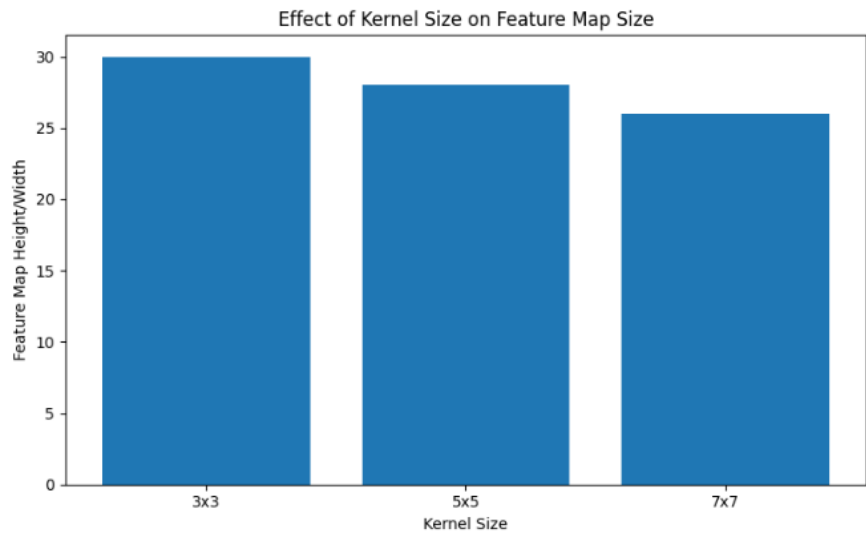


Figure 3: Feature Map Comparison for Different Kernel Sizes

Inference: Increasing the kernel size allows the convolution operation to consider a larger local region of the image. Larger kernels generally produce smaller feature maps when valid padding is used.

9.4 Stride and Padding Comparison

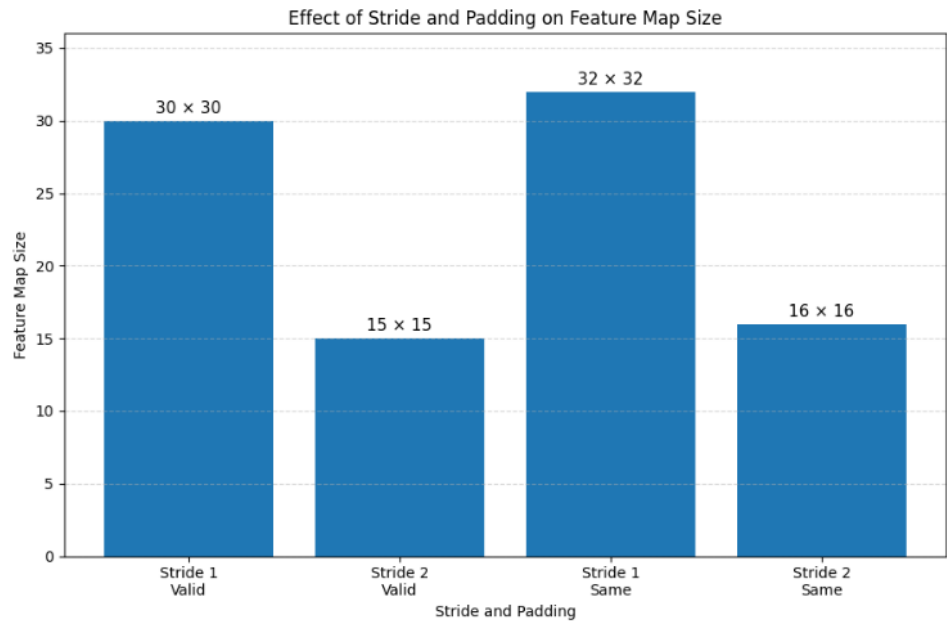


Figure 4: Effect of Stride and Padding on Feature Map Size

Inference: Increasing the stride reduces the spatial dimensions of the feature map. Same padding preserves the spatial dimensions more effectively, whereas valid padding reduces the output size.

9.5 Feature Maps after First Convolution Layer

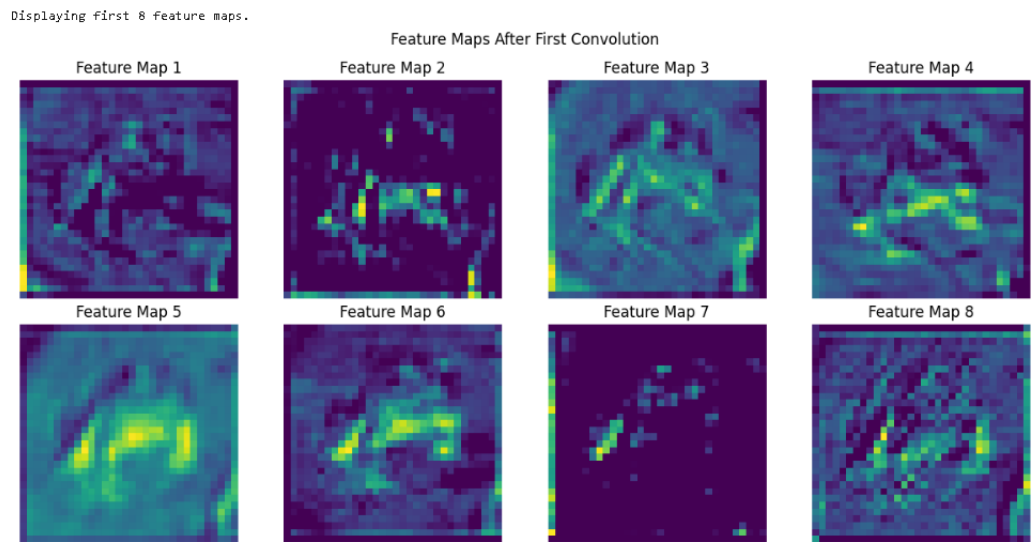


Figure 5: Feature Maps after the First Convolution Layer

Inference: The feature maps show different visual patterns detected by the convolution filters. Early CNN layers generally learn low-level features such as edges, textures, and simple shapes.

9.6 Max Pooling and Average Pooling



Figure 6: Comparison of Max Pooling and Average Pooling

Inference: Max pooling selects the strongest activation from each region, while average pooling computes the average activation. Both methods reduce the spatial dimensions and computational requirements.

9.7 Training and Validation Accuracy

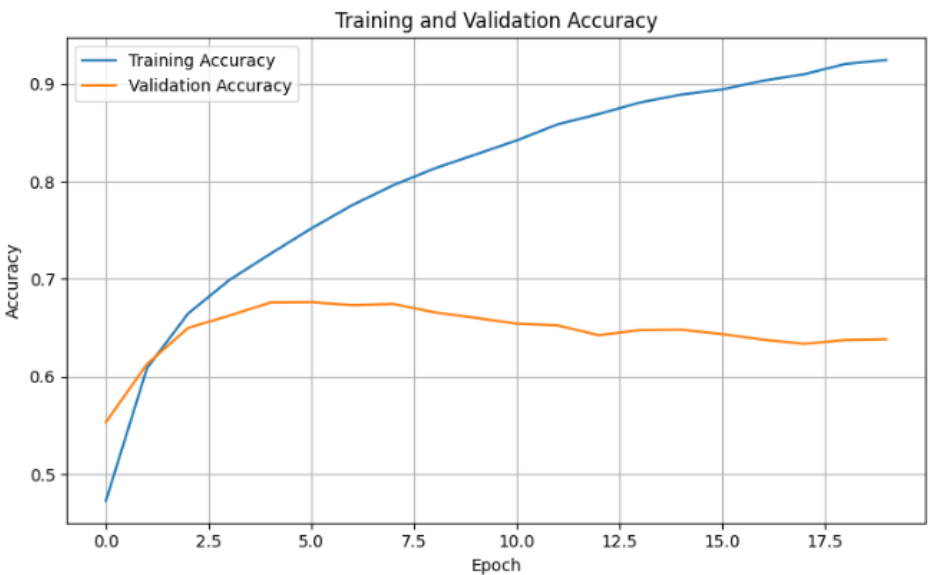


Figure 7: Training and Validation Accuracy

Inference: Training accuracy generally increases as the model learns from the training data. Validation accuracy indicates how well the trained CNN generalizes to unseen data.

9.8 Training and Validation Loss

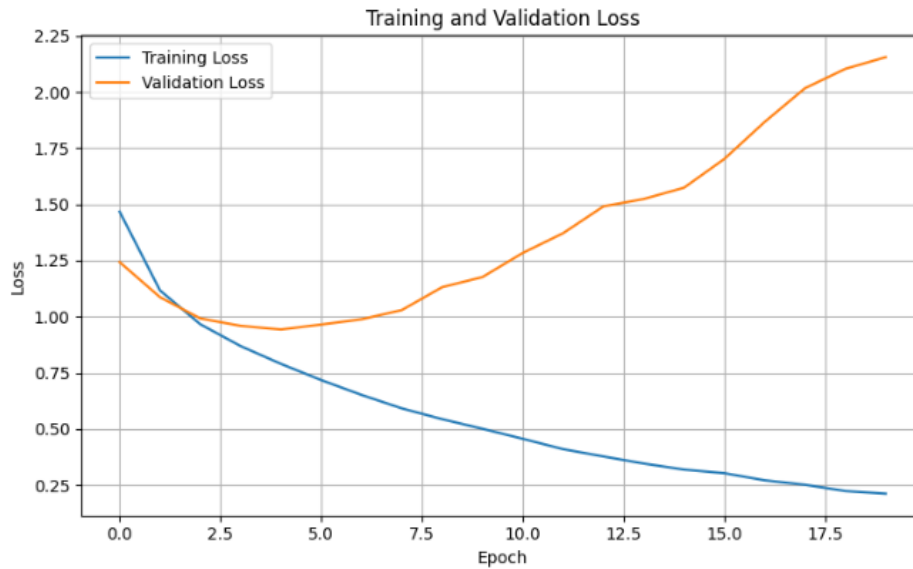


Figure 8: Training and Validation Loss

Inference: The training loss generally decreases as the model learns. A similar decrease in validation loss indicates better generalization, while a growing gap between training and validation loss may indicate overfitting.

9.9 Confusion Matrix

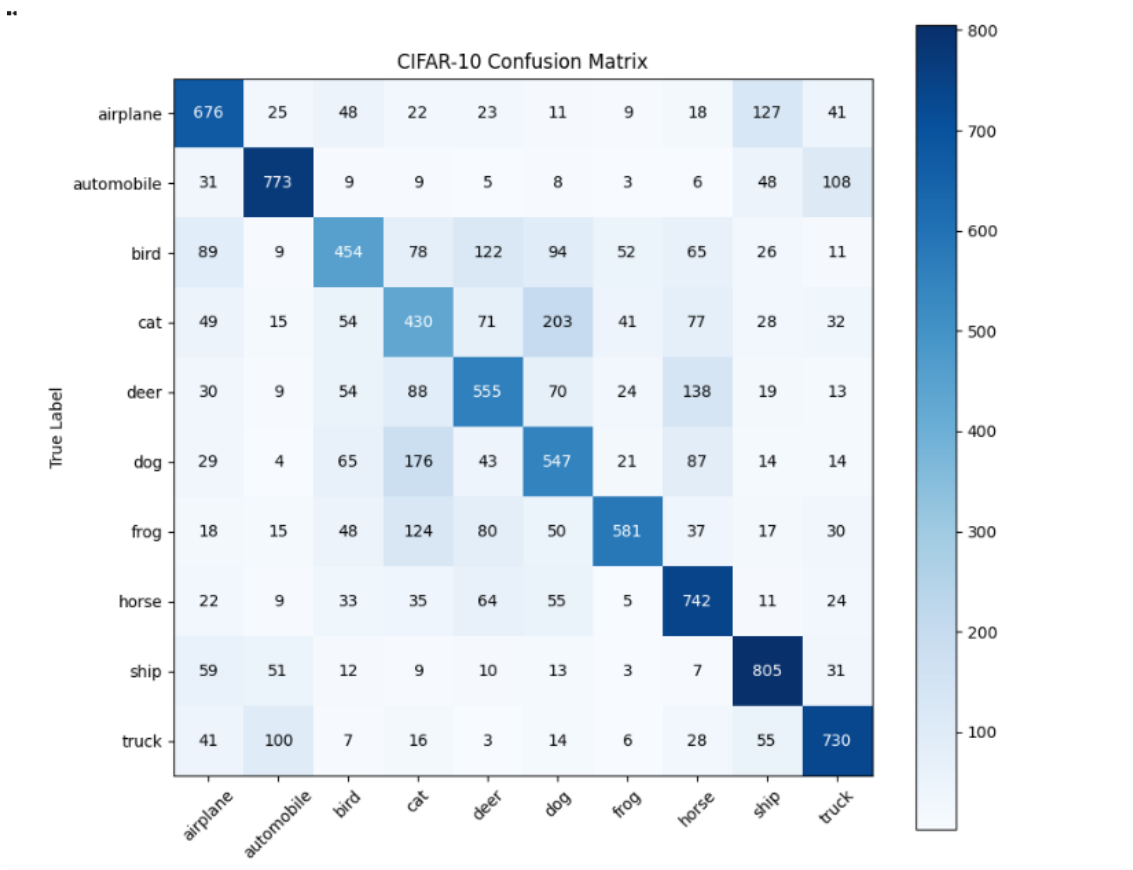


Figure 9: Confusion Matrix for CIFAR-10 Classification

Inference: The confusion matrix shows the classification performance for each CIFAR-10 class. Higher values along the diagonal indicate that the model correctly classified more images.

9.10 Classification Report

Classification Report:				
	precision	recall	f1-score	support
airplane	0.6475	0.6760	0.6614	1000
automobile	0.7653	0.7730	0.7692	1000
bird	0.5791	0.4540	0.5090	1000
cat	0.4357	0.4300	0.4328	1000
deer	0.5686	0.5550	0.5617	1000
dog	0.5136	0.5470	0.5298	1000
frog	0.7799	0.5810	0.6659	1000
horse	0.6158	0.7420	0.6730	1000
ship	0.7000	0.8050	0.7488	1000
truck	0.7060	0.7300	0.7178	1000
accuracy			0.6293	10000
macro avg	0.6311	0.6293	0.6269	10000
weighted avg	0.6311	0.6293	0.6269	10000

Figure 10: Classification Report

Inference: The classification report provides precision, recall, and F1-score for each class. These metrics provide a more detailed evaluation of the CNN beyond overall accuracy.

10. Results

The following results were obtained after training and evaluating the model.

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

```

RESULTS.txt

Final Results
-----
Training Accuracy : 0.9245
Validation Accuracy: 0.6382
Testing Accuracy  : 0.6293
Precision         : 0.6311
Recall           : 0.6293
F1-score         : 0.6269
Parameters       : 268650

```

Figure 11: Model Results

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**
Convolution is preferred because it preserves the spatial structure of images and learns local patterns such as edges, textures, and shapes. Convolutional layers also use shared weights, which greatly reduces the number of parameters compared with fully connected layers.
2. **How does stride affect the feature map size?**
Stride determines how far the convolutional filter moves across the input. A larger stride reduces the spatial dimensions of the feature map. For example, with a 3×3 kernel and valid padding, stride 1 produces a 30×30 feature map from a 32×32 input, while stride 2 produces a 15×15 feature map.
3. **What is the role of padding?**
Padding adds extra pixels, usually zeros, around the boundary of the input image before convolution. It helps preserve spatial dimensions and allows the convolutional filter to process pixels near the image boundaries. With “same” padding, the output size can be maintained for stride 1.
4. **Why is pooling used?**
Pooling reduces the spatial dimensions of feature maps, which decreases computational cost and the number of parameters in subsequent layers. It also helps the network retain important features while making the model less sensitive to small changes in the position of objects.

5. **How do feature maps represent image characteristics?**

Feature maps represent the features detected by convolutional filters. Early layers generally detect simple characteristics such as edges and textures, while deeper layers can detect more complex patterns such as shapes and object parts.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs require fewer parameters because convolutional filters use the same weights across different spatial locations. They also connect each neuron only to a local region of the input instead of connecting it to every pixel. This weight sharing and local connectivity make CNNs much more parameter-efficient than MLPs for image processing.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.

GitHub Repository

The complete source code, implementation files, generated outputs, and requirements for this experiment are available in the following GitHub repository:

GitHub Repository